

Vellum Discord AI Bot – System Plan & Requirements

1. Core Goal

Create an AI-powered Discord bot that manages the Vellum community server, analyzes feedback from users, summarizes useful development insights, and automates server management so the developer does not need to manually monitor Discord discussions.

2. Server Automation

The bot should automatically handle routine server management tasks such as welcoming new users, presenting server rules, onboarding members, guiding users to correct channels, and filtering spam or low-value messages. Key tasks include: - Sending welcome messages - Presenting and enforcing server rules - Assigning member roles after rule acceptance - Redirecting users to correct channels - Creating threads for bug reports - Filtering spam or repeated noise

3. Feedback Monitoring

The bot should monitor selected channels including: - #bug-reports - #feature-requests - #support-chat - #dev-updates Messages in these channels should be collected and stored in a database for analysis.

4. AI Analysis

An AI model processes collected messages and extracts useful development information such as: - Bug reports - Feature requests - User confusion or UX issues - Repeated complaints - Development decisions or updates The AI converts messy discussions into structured information usable by developers.

5. Bug Intelligence System

Bug reports are automatically converted into structured records including: - Bug title - Steps to reproduce - Expected result - Actual result - Platform/device - Severity estimate - Frequency of reports Similar reports are clustered together to detect recurring problems.

6. Daily Development Digest

Every day the bot generates a summary report describing: - Bugs reported that day - Feature requests suggested by users - UX friction or confusion - Most repeated complaints - Suggested development priorities

7. Queryable AI Assistant

The bot should allow queries such as: - "What bugs were reported today?" - "Summarize feature requests this week." - "What problems are users having with publishing?" The bot searches stored messages and generates targeted summaries.

8. Contributor Intelligence System

The bot tracks valuable contributors and assigns a score based on: - Quality bug reports - Feature suggestions - Helping other users - Verified issue reproduction Roles such as Contributor, Tester, or Core Contributor can be automatically assigned based on score thresholds.

9. External Control

The system should allow the developer to access summaries without opening Discord through: - Email summaries - A web dashboard - Messaging integrations This ensures the developer stays informed without constant Discord monitoring.

10. Frontend Dashboard

A web dashboard allows easy interaction with the system. Key dashboard features: - Daily activity reports - Bug clusters - Feature request tracker - Contributor rankings - Server activity metrics

11. System Architecture

Recommended modular architecture: 1. Server Manager Handles welcome messages, rules, onboarding, and moderation helpers. 2. Listener Collects Discord messages from selected channels. 3. AI Analyzer Classifies messages, extracts structured information, clusters duplicates, and generates summaries. 4. Database Stores collected messages and extracted issue records. 5. Dashboard API Provides data to the frontend interface. 6. Frontend Dashboard Displays insights and allows queries.

12. Recommended Technology Stack

Discord Bot: Python (discord.py) AI Runtime: Ollama running a local open-source model
Recommended model: Mistral 7B or similar Database: SQLite (upgrade to PostgreSQL later)
Backend API: FastAPI Frontend: React or Next.js dashboard

13. Long-Term Potential

If developed fully, this system becomes more than a Discord bot. It becomes a community intelligence platform capable of: - analyzing user feedback automatically - prioritizing development work - identifying valuable testers - organizing community discussions - transforming Discord feedback into structured product insights